pdf

https://github.com/SETAPESU26/52_lava-escape

Read a page, added to memory

Here's what the handout and your repo (52_lava-escape) ask for, and a way to get through it in the 60 minutes.

The game: a Pygame vertical platformer. You jump up platforms while lava rises behind you. It has one deliberate bug and three features to add.

Tasks

1. **Fix the bug:** the player snaps to a platform's top surface when jumping up through it from below. Landing should only happen while descending, and only when the player's feet were above the platform's top edge on the previous frame.
2. **Crumbling platforms:** they shake after the player lands, then disappear.
3. **Spring platforms:** they launch the player with extra upward velocity.
4. **Danger HUD and lava surges:** a meter showing the lava's rise speed, plus periodic bursts where the lava speeds up briefly.

Workflow

1. Fork or clone the repo into your personal account. Don't raise a PR to SETAPESU26.
2. Run pip install -r requirements.txt, then python main.py.
3. Record the 10-second "before" video, showing the snapping bug.
4. Prompt the LLM one task at a time, in this order: bug fix, then crumbling, then springs, then HUD and surges. Paste in the relevant file (player.py for the collision, world.py for platforms, game_engine.py for lava and HUD) each time.
5. Record the "after" video showing the fix and all three features.
6. Push to your repo under Lab-4/, with the videos, updated code and exported chat.

Prompting tips (the handout wants 3–4 attempts per task)

- Give the model the exact file and describe the symptom, e.g. “Player lands on a platform while moving upward through it from below.”
- State the rule you want: “Only resolve landing if velocity_y > 0 and the previous frame’s bottom was ≤ the platform’s top.”
- Ask for a minimal diff, and for the new platform types to be added as a kind attribute so the code stays object-oriented.
- Test after each task before moving on, so you know which change broke anything.
- Keep everything in one chat, since you have to export the full history.

I can help in a few ways. If you paste the contents of player.py, world.py and game_engine.py, I can find the bug and write the code for all four tasks. I can also draft the prompts for each task, or help you write the README or submission notes. Which would you like?

```
import pygame

SPEED = 4

class Player:
def __init__(self, x, y):
self.rect = pygame.Rect(x, y, 32, 32)
self.vel_y = 0
self.on_ground = False
self.color = (60,160,220)

def update(self, keys, platforms, width):
dx = 0
if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx = -SPEED
if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx = SPEED
if (keys[pygame.K_SPACE] or keys[pygame.K_w] or keys[pygame.K_UP]) and self.on_ground:
self.vel_y = -13
self.on_ground = False

self.vel_y = min(self.vel_y + 0.55, 12)
self.rect.x = max(0, min(width - self.rect.width, self.rect.x + dx))
self.rect.y += int(self.vel_y)
self.on_ground = False
for p in platforms:
if self.rect.colliderect(p) and self.vel_y > 0 and self.rect.bottom <= p.bottom + 10:
self.rect.bottom = p.top
```

self.vel_y = 0
self.on_ground = True

def draw(self, screen, cam_y):
dr = self.rect.move(0, -int(cam_y))
pygame.draw.rect(screen, self.color, dr, border_radius=6)
pygame.draw.circle(screen,(255,220,180),(dr.centerx, dr.top+8),7) this is player.py

Debugging a collision-detection condition in game movement code.

Debugging a collision-detection condition in game movement code.

Found the bug. It's in the landing check:

python

self.rect.bottom <= p.bottom + 10

This only asks whether the player's feet are above the platform's **bottom** edge (plus a 10px fudge). So when you jump up through a platform from below, the feet end up within that range, the check passes, and the player gets snapped up to p.top. It should instead compare where the feet were on the **previous frame** with the platform's **top** edge.

Fixed player.py

python

```
import pygame


SPEED = 4


class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 32, 32)
        self.vel_y = 0
        self.on_ground = False
        self.color = (60, 160, 220)
```

```
def update(self, keys, platforms, width):
    dx = 0
    if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx = -SPEED
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx = SPEED
    if (keys[pygame.K_SPACE] or keys[pygame.K_w] or keys[pygame.K_UP]) and self.on_ground:
        self.vel_y = -13
        self.on_ground = False

    self.vel_y = min(self.vel_y + 0.55, 12)
    self.rect.x = max(0, min(width - self.rect.width, self.rect.x + dx))

    prev_bottom = self.rect.bottom          # feet position before this frame's move
    self.rect.y += int(self.vel_y)
    self.on_ground = False

    if self.vel_y > 0:                      # only resolve landings while descending
        for p in platforms:
            if self.rect.colliderect(p) and prev_bottom <= p.top:
                self.rect.bottom = p.top
                self.vel_y = 0
                self.on_ground = True
                break                       # stop after first landing

def draw(self, screen, cam_y):
    dr = self.rect.move(0, -int(cam_y))
    pygame.draw.rect(screen, self.color, dr, border_radius=6)
```

pygame.draw.circle(screen, (255, 220, 180), (dr.centerx, dr.top + 8), 7)

What changed

1. prev_bottom is recorded before the vertical move, so landing requires the feet to have been at or above the platform's top, which is the one-way ledge behavior the README describes.
2. The vel_y > 0 check is hoisted out of the loop, so nothing resolves while the player is rising.
3. The + 10 fudge is removed, and a break stops the player from being snapped by two platforms in one frame.

Test it: jump up from directly beneath a platform. You should pass cleanly through it and land on top on the way down.

Prompt for your chat history (for the exported log):

"In player.py, the player snaps to the top of a platform when jumping up through it from below. Landing should only trigger when vel_y > 0 and the player's bottom on the previous frame was above the platform's top. Give me the minimal fix."

The remaining tasks need more than player.py, because platforms are plain pygame.Rects right now and crumbling and spring platforms need a kind and some state. Paste world.py and game_engine.py and I'll write tasks 2 to 4 so they fit together. I'll also give you a small change so player.py can handle spring bounces.

That was only a placeholder. It means the location of the chat PDF once you've saved it, and you don't have the file yet, so first you have to create it.

1. Make the PDF (on the Claude website in your browser):

- Open this conversation and scroll to the bottom so everything has loaded.
- Press Ctrl + P.
- Under **Destination**, choose **Save as PDF**.
- Click **Save** and save it to D:\ as chat.pdf.

Claude is AI and can make mistakes. Please double-check responses.

Claude finished the response